



NATIONAL UNIVERSITY OF MODERN LANGUAGES

Analysis Of Algorithm Presentation Project Report

NAME: Abdul Rafay (FL23791)

Nazish Khan (FL23843)

Ali Atif (FL23824)

Zohaib Ijaz (FL23811)

ROLL NO: FL23791

PROGRAM: BSSE (5th Semester)

COURSE: Analysis Of Algorithm

SUBMITTED TO: Sir Waris Ali

DATE: 9/16/2025

DAY: Tuesday

Longest Common Subsequence and Matrix Chain Multiplication

1. Introduction

Algorithm Design and Analysis is a fundamental subject in computer science that focuses on developing efficient solutions to computational problems. This report covers two important dynamic programming problems: **Longest Common Subsequence (LCS)** and **Matrix Chain Multiplication (MCM)**. Both problems demonstrate how optimal solutions can be obtained by breaking complex problems into simpler overlapping subproblems.

2. Longest Common Subsequence (LCS)

2.1 Definition

The **Longest Common Subsequence (LCS)** problem involves finding the longest sequence that appears in the same order (not necessarily contiguously) in two given sequences. It is commonly applied to string comparison problems.

Example:

- Sequence 1: ABC
- Sequence 2: CAB
- LCS: AB

2.2 Importance of LCS

The LCS problem is widely used in:

- File comparison tools
- Version control systems
- DNA sequence analysis
- Text similarity checking

2.3 Dynamic Programming Approach

To solve the LCS problem efficiently, a **dynamic programming table** is used. The idea is to store results of subproblems so that they are not recomputed.

Conditions Used

- If $i = 0$ or $j = 0$, then $C[i][j] = 0$
- If $x_i = y_j$, then
 $C[i][j] = C[i-1][j-1] + 1$
- Otherwise,
 $C[i][j] = \max(C[i-1][j], C[i][j-1])$

2.4 Example

Given Sequences:

- Seq1: ABCDEFG
- Seq2: CDBAEF

Example: Find the length of the longest subsequence present in both of them.....

Seq1: ABCDEFG, Seq2: CDBAEF

	Seq1	A	B	C	D	E	F	G
Seq2	0	0	0	0	0	0	0	0
C	0	0	0	1	1	1	1	1
D	0	0	0	1	2	2	2	2
B	0	0	1	1	2	2	2	2
A	0	1	1	1	2	2	2	2
E	0	1	1	1	2	3	3	3
F	0	1	1	1	2	3	4	4

C D E F

2.5 Algorithm for LCS

1. Find lengths of both sequences
2. Create two matrices B and C
3. Initialize first row and column with zero
4. Compare characters of both sequences
5. Fill the table using recurrence relations
6. Return the length and sequence of LCS

3. Matrix Chain Multiplication (MCM)

3.1 Definition

Matrix Chain Multiplication is the problem of finding the **most efficient way to multiply a chain of matrices** by choosing an optimal order of parenthesization.

Important Note:

Matrix multiplication is **associative but not commutative**, meaning the order of multiplication affects the cost.

3.2 Objective

The main goal of MCM is to **minimize the total number of scalar multiplications**, not to change the result of the multiplication.

3.3 Applications of MCM

- Compiler design (code optimization)
- Database query optimization
- Scientific computing

3.4 Cost Calculation

If matrix A has dimensions $(p \times q)$ and matrix B has dimensions $(q \times r)$, then cost of multiplication = $p \times q \times r$.

3.5 Example

Given matrices:

- $A_1 = (5 \times 4)$
- $A_2 = (4 \times 6)$
- $A_3 = (6 \times 2)$

Two possible ways:

1. $(A_1 A_2) A_3 \rightarrow \text{Cost} = 180$
2. $A_1 (A_2 A_3) \rightarrow \text{Cost} = 88$

Optimal Solution: $A_1 (A_2 A_3)$

3.6 Dynamic Programming Formula

Let $m[i][j]$ represent the minimum cost of multiplying matrices from A_i to A_j .

$$m(i, j) = \min_{i \leq k < j} \{m(i, k) + m(k + 1, j) + p_{i-1} \times p_k \times p_j\}$$

Base Case:

$$m(i,i) = 0$$

3.7 Parenthesization Count

The number of ways to parenthesize n matrices is given by the **Catalan number** formula.

3.8 Final Result

Using dynamic programming tables m (cost table) and s (split table), the minimum cost and optimal parenthesization are determined efficiently.

Optimal Parenthesization Example:

$$((A_1(A_2A_3))A_4)$$

4. Comparison of LCS and MCM

Feature	LCS	MCM
Technique	Dynamic Programming	Dynamic Programming
Goal	Maximize sequence length	Minimize cost
Input	Sequences	Matrix dimensions
Output	LCS length/sequence	Optimal parenthesization

5. Conclusion

Both **Longest Common Subsequence** and **Matrix Chain Multiplication** are classic examples of dynamic programming problems. They show how complex problems can be solved efficiently by storing intermediate results. Understanding these algorithms helps in building a strong foundation for advanced topics in algorithm optimization and software development.